

SGX Source Code Documentation (Phase 1)

Date: 19 Jan, 2026

Project: SGX (Phase 1)

Introduction:

The SG-X Guardian Client is a Zero-Trust Edge Security Agent designed for robust protection of distributed IoT and enterprise nodes. It ensures continuous security by operating autonomously, enforcing policies, performing and maintaining secure peer-to-peer communication for critical functions that persist even during network outages or cloud disconnection.

SGX provides a enforced Trusted Execution Environment that protects SG-X Guardian's most critical security operations from the host operating system and other privileged software. It ensures cryptographic keys, policy enforcement logic, and security intelligence remain confidential and tamper-resistant even if the device is compromised. SGX enables remote attestation, allowing Guardian to prove its integrity and authenticity in untrusted or hostile environments.

1. SGX Project Structure:



```
graph TD
    SGX[SGX] --> github[.github]
    SGX --> build[build]
    SGX --> config[config]
    SGX --> docs[docs]
    SGX --> logs[logs]
    SGX --> packaging[packaging]
    SGX --> policies[policies]
    SGX --> proto[proto]
    SGX --> schemas[schemas]
    SGX --> sgx-agent[sgx-agent]
    SGX --> sgx-pa-cli[sgx-pa-cli]
    SGX --> src[src]
    SGX --> target[target]
    SGX --> tests[tests]
    SGX --> gitignore[.gitignore]
    SGX --> build_rs[build.rs]
    SGX --> cargo_lock[Cargo.lock]
    SGX --> cargo_toml[Cargo.toml]
    SGX --> deny_toml[deny.toml]
```

1.1. .github:

GitHub Actions Workflows:

- Continuous Integration (CI) Continuous Deployment (CD)
- Security scanning (CodeQL)

1.2. config:

Config Folder provides the details of Nodes e.g, node_id, hostname, port, ip.

1.3. Docs:

Docs folder provides the detail of the decision logs, api specifications, cicd pipeline documentation etc.

1.4. Logs:

When a node is active, it continuously generates runtime logs that provide visibility into its operational status.

1.5. Packaging:

The packaging/ directory defines how the SGX Guardian Client is packaged and integrated with the operating system, enabling standardized installation and service management across Linux distributions.

1.6. policies:

When the nodes are running, the policy folder ensures that what policy is currently running and which policy is in backup:

- Active Policy
- Backup Policy

1.7. Proto:

Proto handles peer discovery and mDNS handshake, perform health and latency checks, handles signed policy distribution and validation. All proto files are compiled during the build step.

1.8. Schemas:

This folder contains UEP policy and what operations nodes will perform. Admin update and sign this UEP policy and pass to the nodes.

1.9. Sgx-agent:

It contains certificates and keys of the nodes.

1.10. sgx-pa-cli:

From here, admin can generate keys, sign policy, check recent peers attestation and check logs.

1.11. src:

This folder contains all the core business logic of SG-X, implementing the main functionality, workflows, and operational rules of the system. It serves as the central location for the application's executable code.

1.12. Target:

Compiled binaries and build artifacts generated by Cargo (Rust build system).

1.13. Tests:

It performs automated test covering policy validation signing, verification and core runtime.

1.14. .gitignore:

In this file we tell the project that what confidential files should not be push to git.

1.15. build.rs

Rust build script executed at compile time. Used to generate code, compile protobufs, or perform environment-specific build steps required by the SGX.

1.16. Cargo.lock

Ensures reproducible and deterministic builds across environments and systems.

1.17. Cargo.toml

Defines the project metadata, dependencies, features and workspace configuration for SGX Guardian.

1.18. Deny.toml

Used to enforce dependency policies:

- License Compliance
- Duplicate Dependencies
- Security Advisories

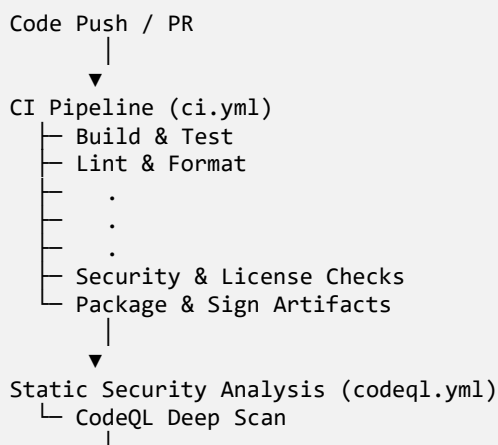
2. Source Code Overview:

github/workflows:

This directory defines automation workflows responsible for validating, securing, and governing the SGX Guardian codebase.

```
.github/workflows/  
├── ci.yml  
└── codeql.yml
```

High-Level Workflow Flow (Diagram)



2.1. 📄 ci.yml — Continuous Integration Pipeline

1. Purpose

Ensure every code change is buildable, tested, secure, and compliant before release or merge.

2. Responsibility

- Compile Rust source code
- Run tests, linting, and formatting checks
- Perform SAST, dependency, and license audits
- Generate coverage reports
- Package and cryptographically sign release artifacts

3. Inputs

- Rust source code (src/, sgx-agent/)
- Protobuf definitions (proto/)
- Cargo manifests (Cargo.toml, Cargo.lock)

4. Outputs

- Validated build results
- Code coverage reports
- Uploaded build artifacts for verification or deployment

5. Security / Constraints

- Fails build on any warning or audit issue
- Enforces license and dependency policies
- Uses GPG signing to ensure artifact integrity
- Runs only on protected branches (main) and on open PR

Concept Clarifying Code Snippet (CI)

```
- name: Run Clippy (linting)
  run: cargo clippy -- -D warnings
```

2.2. codeql.yml — Static Application Security Testing (SAST)

1. Purpose

Detect code-level security vulnerabilities using GitHub CodeQL before they reach production.

2. Responsibilities

- Build project for analysis
- Perform deep static analysis on Rust code
- Detect insecure data flows, logic flaws, and API misuse
- Publish findings to GitHub Security dashboard

3. Inputs

- Compiled Rust source code
- Protobuf-generated stubs
- CodeQL Rust security queries

4. Outputs

- Security alerts with severity levels
- Line-level vulnerability reports
- Historical security tracking in GitHub

5. Security / Constraints

- Requires full build before analysis (accurate data flow)
- Security findings are immutable audit records
- Runs with minimal permissions (principle of least privilege)

6. Concept Clarifying Code Snippet (CodeQL)

```
- name: Initialize CodeQL
  uses: github/codeql-action/init@v3
  with:
    languages: rust
```

2.3. config/ — Node Configuration Files (nodeA.yaml, nodeB.yaml, nodeC.yaml)

nodeA.yaml / nodeB.yaml / nodeC.yaml



Config Loader
(src/config_loader.rs)



Runtime Node Context
(identity, address, keys)



Attestation Service
Networking (gRPC)
Policy Enforcement

7. Purpose

Define static identity and network details for each SGX Guardian node participating in the system.

8. Responsibility

- Identify a node uniquely
- Provide network binding (IP + port)
- Expose cryptographic identity (public key reference)
- Enable multi-node simulation and trust relationships

9. Inputs

Example (nodeA.yaml):

```
node_id: "nodeA"  
hostname: "guardian-node-A"  
ip: "127.0.0.1"  
port: 50051  
public_key: "placeholder-key-A"
```

- Used as input by:
- Config loader at startup
- Attestation service
- gRPC server/client initialization

10. Outputs

- After loading, the config produces:
- In-memory NodeContext structure
- Bound gRPC listener on defined IP/port
- Node identity available to:
- Attestation flow
- Peer communication
- Policy decisions

11. Security / Constraints

- node_id must be unique across nodes
- port conflicts will prevent startup
- public_key must match signed policies / attestations

- Config files are trusted inputs (should be protected at rest)

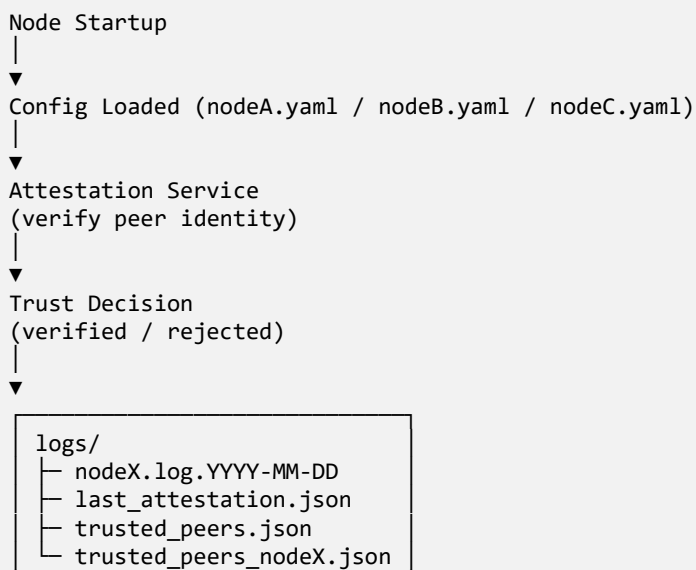
🔍 Concept Clarifying Code Snippet (CodeQL)

```
let node_cfg =
  ConfigLoader::load("config/nodeA.yaml");
  attestation_service.init(node_cfg.node_id,
    node_cfg.public_key);
```

2.4. 📁 logs/ — Runtime Logs & Trust State

This directory contains runtime-generated evidence of node behavior, peer trust, and attestation results.

High-Level Flow (Diagram)



1. Purpose

Provide audit-grade visibility into node activity, peer attestation, and trust decisions during runtime.

2. Responsibility

- Record node execution events
- Persist attestation outcomes
- Maintain list of verified (trusted) peers
- Support debugging, forensics, and audit review

3. Inputs

- Sources of data:
- Node configuration (config/node*.yaml)
- Incoming peer connections (gRPC)
- Attestation verification results
- Policy signature checks

Example input (from runtime):

```
{
  "ip": "127.0.0.1:50151",
  "peer_id": "127.0.0.1:50151",
  "status": "verified",
  "timestamp": "2026-01-22T05:31:48Z"
}
```

4. Outputs

Log Artifacts Generated:

nodeA.log.YYYY-MM-DD

- → Node-specific runtime activity (startup, connections, errors)
 - last_attestation.json
- → Most recent attestation result
 - trusted_peers.json
- → Global list of verified peers
 - trusted_peers_nodeA.json
- → Node-scoped trusted peer view
 - These files represent the current trust state of the system.

5. Security / Constraints

- Logs are append-only (no runtime mutation)
- trusted_peers*.json is written only after successful attestation
- Unverified peers are never persisted as trusted
- Log files are sensitive artifacts and must be protected at rest
- Used as forensic evidence during incident analysis

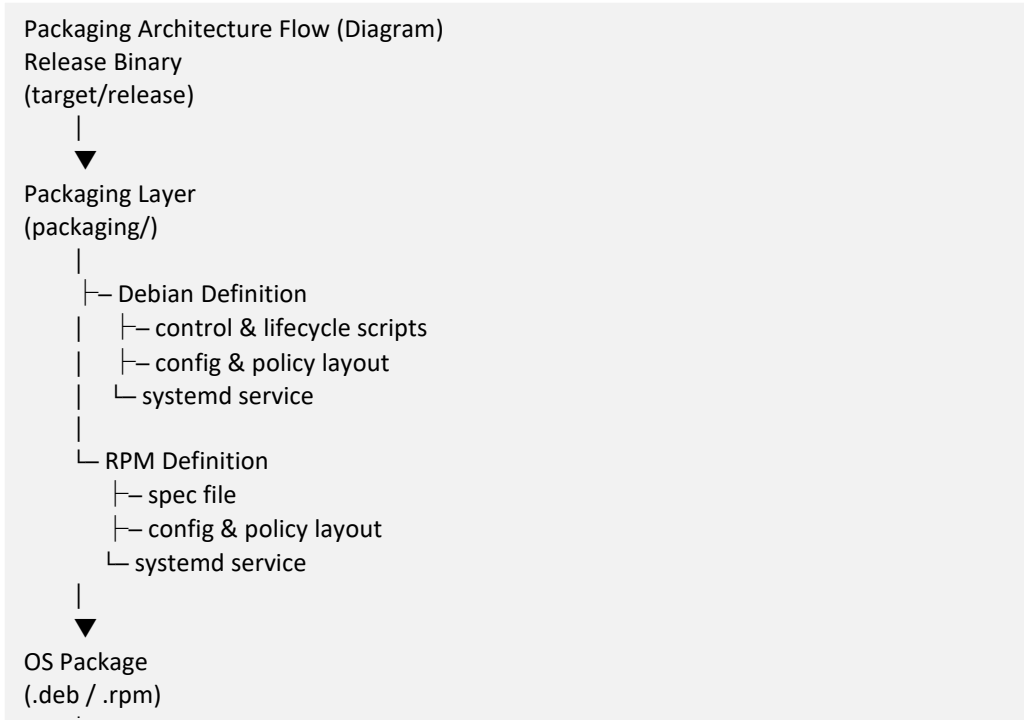
6. Concept-Clarifying Code Snippet

```
if attestation_result.is_verified() {  
    trusted_peers.store(peer_id,  
timestamp);  
    log::info!("Peer {} marked as  
trusted", peer_id);  
}
```

2.5. packaging/ — OS Packaging & Service Integration Layer Folder & File Structure

```
packaging/  
├── deb/  
│   ├── config/  
│   │   ├── nodeA.yaml  
│   │   ├── nodeB.yaml  
│   │   └── nodeC.yaml  
│   ├── policies/  
│   │   ├── active_policy.yaml  
│   │   ├── backup_policy.yaml  
│   │   └── policy.sig  
│   ├── DEBIAN/  
│   │   ├── control  
│   │   ├── postinst  
│   │   ├── prerm  
│   │   └── postrm  
│   ├── scripts/  
│   │   ├── install_layout.sh  
│   │   ├── permissions.sh  
│   │   └── sanity_check.sh  
│   └── systemd/  
│       └── sgx-guardian.service  
└── rpm/  
    ├── config/  
    │   ├── nodeA.yaml  
    │   ├── nodeB.yaml  
    │   └── nodeC.yaml  
    ├── policies/  
    │   ├── active_policy.yaml  
    │   ├── backup_policy.yaml  
    │   └── policy.sig  
    ├── SOURCES/  
    ├── SPECS/  
    │   └── sgx-guardian-client.spec  
    └── systemd/  
        └── sgx-guardian.service
```

Packaging Architecture Flow (Diagram)



1. Purpose

The packaging/ directory defines how the SGX Guardian Client is packaged and integrated with the operating system, enabling standardized installation and service management across Linux distributions.

2. Responsibility

packaging/ is responsible for:

- Defining OS-specific package metadata (Debian / RPM)
- Declaring filesystem layout for configuration and policy files
- Registering the SGX Guardian systemd service
- Handling install, upgrade, and removal lifecycle hooks
- Ensuring consistent deployment behavior across platforms
- It contains no runtime logic and no security decision-making code.

3. Inputs

Inputs consumed by the packaging layer include:

- Pre-built SGX Guardian Client binary
- Configuration file templates
- Policy artifacts (active, backup, signature)
- systemd service definition

Packaging metadata files:

- DEBIAN/control
- sgx-guardian-client.spec

4. Outputs

The packaging process produces:

OS-native installable packages:

- Debian (.deb)
- RPM (.rpm)

Installed system components:

- SGX Guardian binary
- Configuration and policy file layout
- Registered systemd service unit
- These outputs enable the SGX Guardian Client to be managed as a standard OS service.

5. Security / Constraints

- Packaging does not generate or modify trust material
- No private keys are embedded in packages
- Policies included are subject to runtime verification
- Service execution follows OS-level permission controls
- Packaging logic assumes a pre-initialized system environment
- Debian and RPM packages share identical security posture

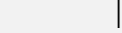
2.6. policies/ — Policy Definition & Enforcement

This directory contains security policies that control network and behavior rules enforced by SGX Guardian nodes.

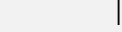
```
policies/  
├── active_policy.yaml  
├── backup_policy.yaml  
└── policy.sig
```

High-Level Policy Flow (Diagram)

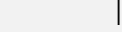
Admin Defines Policy



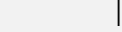
Policy YAML Created
(active_policy.yaml)



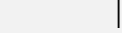
Admin Signs Policy
(policy.sig)



Policy Verification
(signature + schema)



Active Policy Loaded
(Runtime Enforcement)



Traffic / Requests
Allowed or Denied

1. Purpose

Define authoritative security rules that determine what network traffic and actions are allowed or denied on each node.

2. Responsibility

- Store the currently enforced policy
- Keep a safe fallback policy
- Ensure only signed and verified policies are applied
- Provide a deterministic rule set for all nodes

3. Inputs

Policy Definition (active_policy.yaml)

```
policy_id: "123e4567-e89b-12d3-a456-426614174000"
version: "1.0.0"
rules:
  - id: "rule-001"
    action: "ALLOW"
    src: "10.0.0.0/24"
    dst: "0.0.0.0/0"
    protocol: "TCP"
    port: 443
```

- Signature Input
- policy.sig (generated by admin CLI)
- Admin public key (guardian_public.key)

4. Outputs

- active_policy.yaml
 - → Policy currently enforced by all running nodes
- backup_policy.yaml
 - → Last known good policy (used on failure / rollback)
- Runtime effects:
 - Traffic allowed or blocked
- Decisions logged
 - Policy ID referenced in audits

5. Security / Constraints

- Policies must be signed by a trusted admin key
- Unsigned or tampered policies are rejected
- Only one active policy can exist at runtime
- Backup policy is read-only during normal operation
- Policy schema must pass validation (schemas/)

Concept-Clarifying Code Snippet

```
if verify_signature(policy, policy_sig,
admin_pubkey)? {
  policy_engine.load_active(policy);
} else {
  policy_engine.load_backup();
}
```

2.7. proto/ — Network Communication Contracts (Combined)

This directory defines the authoritative protobuf contracts used for node discovery, health checks, attestation context, and policy exchange across the SGX Guardian system.

```
proto/
├── peer.proto
├── ping.proto
└── policy.proto
```

High-Level Combined Flow (Diagram)



1. Purpose

Provide a single, consistent network contract layer that enables SGX Guardian nodes to:

- Identify each other
- Verify liveness
- Exchange policy data securely

2. Responsibility

- Collectively, the proto definitions are responsible for:
- Describing node identity and status (PeerInfo)
- Performing liveness and connectivity checks (Ping)
- Transporting signed security policies (PolicyInfo)
- Acting as the foundation for attestation and trust decisions

3. Inputs

- Runtime-generated inputs include:
- Node identity (node ID, hostname, IP)
- Cryptographic public keys
- Node status and software version
- Policy metadata, payload, and signature
- Health-check requests from peers

Example combined input:

- PeerInfo + PingRequest + PolicyInfo
- These inputs originate from:
- Node configuration
- Admin policy tooling
- gRPC peer connections

4. Outputs

- The proto layer produces:

- Serialized gRPC messages exchanged between nodes
- Verified policy payloads forwarded to the policy engine
- Peer identity data forwarded to attestation logic
- Health-check responses used by monitoring and CLI tools
- All higher-level components rely on these outputs for correct runtime behavior.

5. Security / Constraints

- Proto messages define structure only, not trust
- Identity claims (PeerInfo) are verified later via attestation
- Ping responses do not imply trust
- Policies are applied only after signature verification
- Any malformed or unsigned message is rejected at validation stage

Concept-Clarifying Code Snippet

```
// Combined proto usage example
let peer = PeerInfo { .. };
client.ping(PingRequest { from: peer.node_id }).await?;
verify_policy_signature(&policy_info)?;
```

2.8. schemas/ — Policy Validation Schema

This directory contains schema definitions used to validate security policies before they are signed, distributed, or enforced.

schemas/
└─ uep_policy_v1.yaml

High-Level Flow (Diagram)

```
Admin Updates UEP Policy
(UEP rules: ALLOW / DENY)
↓
Schema Validation
(schemas/uep_policy_v1.yaml)
↓
sgx-pa-cli
(validate + sign)
↓
Generated Signature
(policy.sig)
↓
Policy Delivered to Runtime Load Location
(node policy load directory)
↓
Policy Engine
(active / backup selection)
↓
Runtime Enforcement
+ Logs / Audit
```

1. Purpose

Define the authoritative structure and rules for security policies to ensure they are well-formed, complete, and predictable.

2. Responsibility

- Enforce required fields (policy_id, version, rules)
- Validate rule structure (ALLOW / DENY, src, dst, protocol, port)
- Prevent malformed or incomplete policies from being accepted
- Act as the first gate before cryptographic verification

3. Inputs

Validated input example:

```
policy_id:"123e4567-e89b-12d3-a456-426614174000"
version: "1.0.0"
rules:
- id: "rule-001"
  action: "ALLOW"
  src: "10.0.0.0/24"
  dst: "0.0.0.0/0"
  protocol: "TCP"
  port: 443
```

Input source:

- Admin-defined policy files
- Policy updates pushed to nodes

4. Outputs

Validation success → policy allowed to proceed for signing / enforcement

Validation failure → policy rejected immediately

Downstream consumers:

- Policy signing logic
- Policy engine
- Attestation and enforcement flow

5. Security / Constraints

- Schema validation happens before signature verification
- Invalid structure is rejected even if signed
- Prevents logic ambiguity and unsafe defaults
- Schema versioning (v1) enables safe future evolution

Concept-Clarifying Code Snippet

```
validate_against_schema(policy_yaml,
"uep_policy_v1.yaml"?);
```

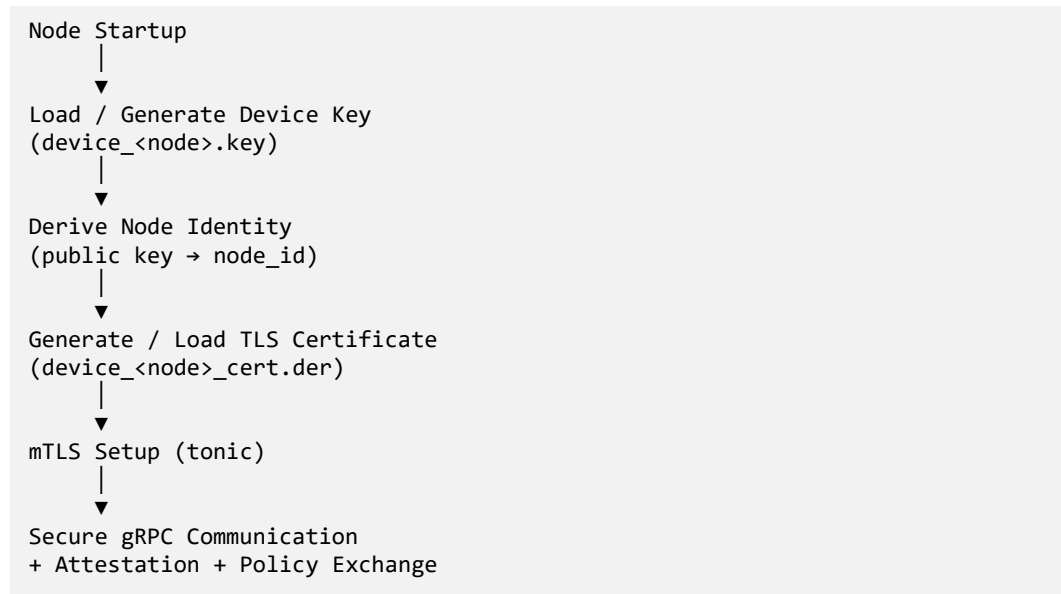
2.9. **sgx-agent/** — Node Identity, Keys & TLS Material

This directory contains node-specific cryptographic material used to establish identity, attestation, and mutual TLS (mTLS) between SGX Guardian nodes.

sgx-agent/

```
├── device_nodeA.key
├── device_nodeA_cert.der
├── device_nodeB.key
├── device_nodeB_cert.der
├── device_nodeC.key
└── device_nodeC_cert.der
```

High-Level Identity & TLS Flow (Diagram)



1. Purpose

Provide a persistent cryptographic identity for each SGX Guardian node and enable secure, mutually authenticated communication.

2. Responsibility

- Store node private keys securely
- Bind node identity to cryptographic material
- Generate and manage node TLS certificates
- Enable mTLS for gRPC server and client communication
- Support attestation and trust establishment

3. Inputs

Node identifier (nodeA, nodeB, nodeC)

Generated or existing private key:

- device_<node>.key
- Node hostname and IP (used as SAN)
- Policy schema and attestation inputs (indirectly)

4. Outputs

Node-specific TLS certificate:

- device_<node>_cert.der
- Public key material (device_public.der)
- Attestation and enforcement flow

In-memory TLS identity used by:

- gRPC server
- gRPC client
- Attestation service

These outputs enable encrypted and authenticated node-to-node communication.

5. Security / Constraints

- Private keys must never leave the node
- Keys are generated once and reused across restarts
- Certificates are regenerated only if missing or invalid
- SAN entries bind certificate to node hostname/IP
- Compromised keys require manual rotation
- Files must be protected at rest (filesystem permissions)

Concept-Clarifying Code Snippet

```
let key_path = format!("/var/lib/sgx-guardian/sgx-agent/device_{}.key", node_id);
let cert_path = format!("/var/lib/sgx-guardian/sgx-agent/device_{}_cert.der", node_id);

tls::ensure_node_certificate_or_generate(&key_path,
&cert_path, &san)?;
```

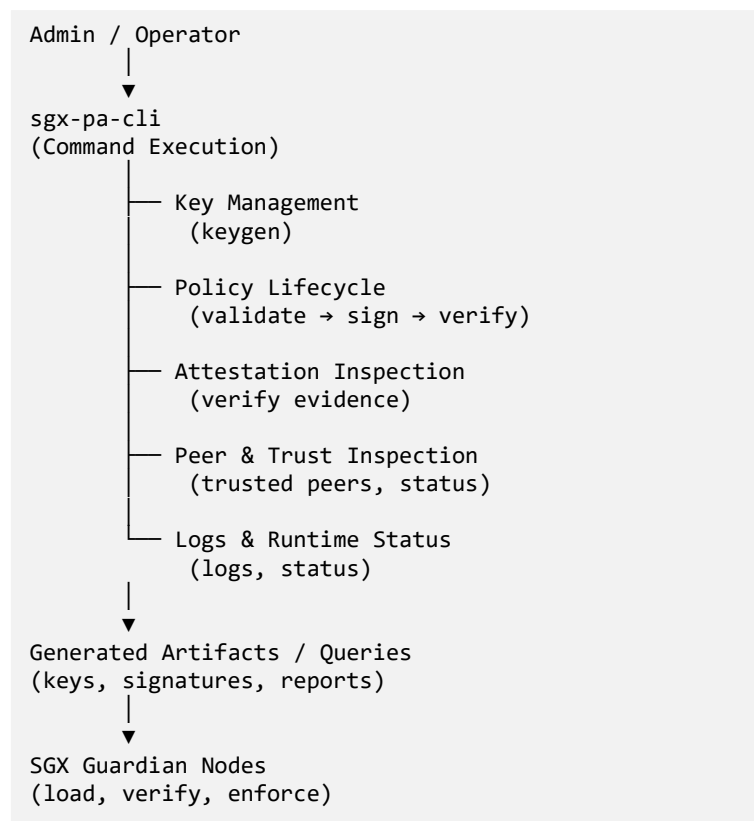
2.10. **sgx-pa-cli** — Policy & Administration Command-Line Interface

The SGX Policy & Administration CLI (sgx-pa-cli) is a trusted operator tool used to manage keys, policies, attestation verification, peer trust, and runtime inspection of SGX Guardian nodes.

Internal Structure Overview

```
sgx-pa-cli/
├── src/
│   ├── main.rs           → CLI entrypoint & command routing
│   ├── config.rs         → CLI configuration loading
│   ├── lib.rs            → Shared exports
│   └── policy_signer.rs  → Policy digest + signing logic
│       └── commands/
│           ├── keygen.rs   → Admin key generation
│           ├── sign.rs     → Policy signing
│           ├── verify.rs   → Policy verification
│           ├── attestation.rs → Attestation verification
│           ├── peers.rs    → Trusted peer inspection
│           ├── logs.rs     → Runtime log inspection
│           ├── status.rs   → Node status queries
│           └── mod.rs      → Command registry
├── tests/                → CLI behavior & security tests
├── guardian_private.key  → Admin signing key
└── guardian_public.key   → Admin verification key
```

High-Level CLI Flow (Diagram)



1. Purpose

Provide a secure, auditable, and deterministic interface for administrators to:

- Authorize policies
- Manage cryptographic trust
- Inspect node state
- Verify security evidence

2. Responsibility

- sgx-pa-cli is responsible for:
- Generating and managing admin signing keys
- Validating policies against schemas
- Cryptographically signing policies
- Verifying policy integrity and signatures
- Verifying attestation evidence
- Inspecting trusted peers and runtime logs
- Acting as the only trusted policy-authority interface

3. Inputs

Node identifier (nodeA, nodeB, nodeC)

Inputs accepted by the CLI include:

Generated or existing private key:

- UEP policy files (YAML)
- Policy schemas
- Attestation evidence blobs
- Runtime log files
- Node trust data (trusted_peers.json)

- Admin private/public keys

Example CLI input:

```
sgx-pa-cli sign --policy active_policy.yaml --key  
guardian_private.key
```

4. Outputs

Depending on the command, the CLI produces:

- policy.sig (signed policy artifact)
- Verification reports (success / failure)
- Human-readable status output
- Parsed logs and trust views
- Cryptographic digests (policy hash)

These outputs are consumed by:

- SGX Guardian nodes
- Auditors
- CI / security reviews

5. Security / Constraints

- Private admin key never leaves CLI
- Policies must pass schema validation before signing
- Tampered or unsigned policies are rejected
- CLI enforces deterministic policy digests
- Output artifacts are immutable
- CLI commands are read-only unless explicitly signing or generating keys

Concept-Clarifying Code Snippet

```
// main.rs (simplified)
match command {
  Commands::Sign { policy, key } => {
    let digest =
PolicySigner::digest(&policy)?;
    PolicySigner::sign(&digest, &key)?;
  }
}
```

2.11. src/ — SGX Guardian Client

The `src/` directory implements the entire trusted runtime of the SGX Guardian Client. It is responsible for identity, trust establishment, secure communication, policy enforcement, auditing, and observability.

COMPLETE END-TO-END RUNTIME FLOW (Single Diagram)



1. Purpose

The `src/` folder implements a zero-trust, policy-driven runtime agent that:

- Establishes cryptographic node identity
- Verifies peer trust via attestation
- Enforces signed security policies
- Secures communication using mTLS

- Records tamper-evident audit evidence
- Exposes operational metrics
- Continues safely under failure conditions

2. Responsibility

- Collectively, the src/ directory is responsible for:
- Bootstrapping node identity and cryptographic material
- Discovering peers and validating them
- Enforcing admin-authorized policies
- Securing all inbound and outbound traffic
- Persisting audit-grade evidence
- Providing observability (logs + metrics)
- Maintaining runtime safety and determinism

3. Inputs

The runtime consumes the following inputs:

- Configuration & Identity
- Node YAML configs (nodeA.yaml, etc.)
- Persistent node private keys
- TLS certificate material
- Trust & Policy
- Signed policy artifact (policy.sig)
- Policy schemas
- Admin public key
- Runtime Signals
- gRPC requests
- Peer discovery messages
- Environment variables (cloud, metrics)

4. Outputs

The runtime produces:

Depending on the command, the CLI produces:

- Secure gRPC services
- Enforced system rules (e.g., nftables)
- Trusted peer state
- Structured logs
- Tamper-evident audit logs
- Metrics endpoints
- Optional cloud heartbeat signals

These outputs are consumed by:

- SGX Guardian nodes
- Auditors
- CI / security reviews

5. Security / Constraints

- Identity $\neq$ Trust (attestation mandatory)
- Policies must be schema-valid AND signed
- Only one active policy at a time
- Backup policy always preserved

- Private keys never leave node
- Audit logs are append-only and hash-chained
- Runtime never fails open

6. INTERNAL MODULE ARCHITECTURE (Mapped)

Identity & Crypto:

- key_manager.rs — key load/generation
- tls.rs — certificate + mTLS setup

Trust Establishment

- p2p_discovery.rs — peer detection
- attestation_service.rs — evidence creation & verification

Policy System

- policy.rs — parse & validate
- policy_manager.rs — verify & activate
- policy_state.rs — active/backup state

Enforcement

- enforcement/model.rs — policy intent
- enforcement/translator.rs — system rules
- enforcement/validator.rs — safety checks
- enforcement/executor.rs — apply rules

Networking

- server.rs — gRPC server
- client.rs — gRPC client
- proto/sgx.rs — generated contracts

Audit & Logging

- audit/event.rs — event model
- audit/haser.rs — hash chain
- audit/writer.rs — append-only writer
- audit/verifier.rs — integrity check
- logging.rs — structured runtime logs

Observability

- metrics.rs — counters & state
- metrics_server.rs — metrics endpoint

Cloud (Optional)

- cloud/client.rs — outbound heartbeat
- cloud/mock_server.rs — dev/testing

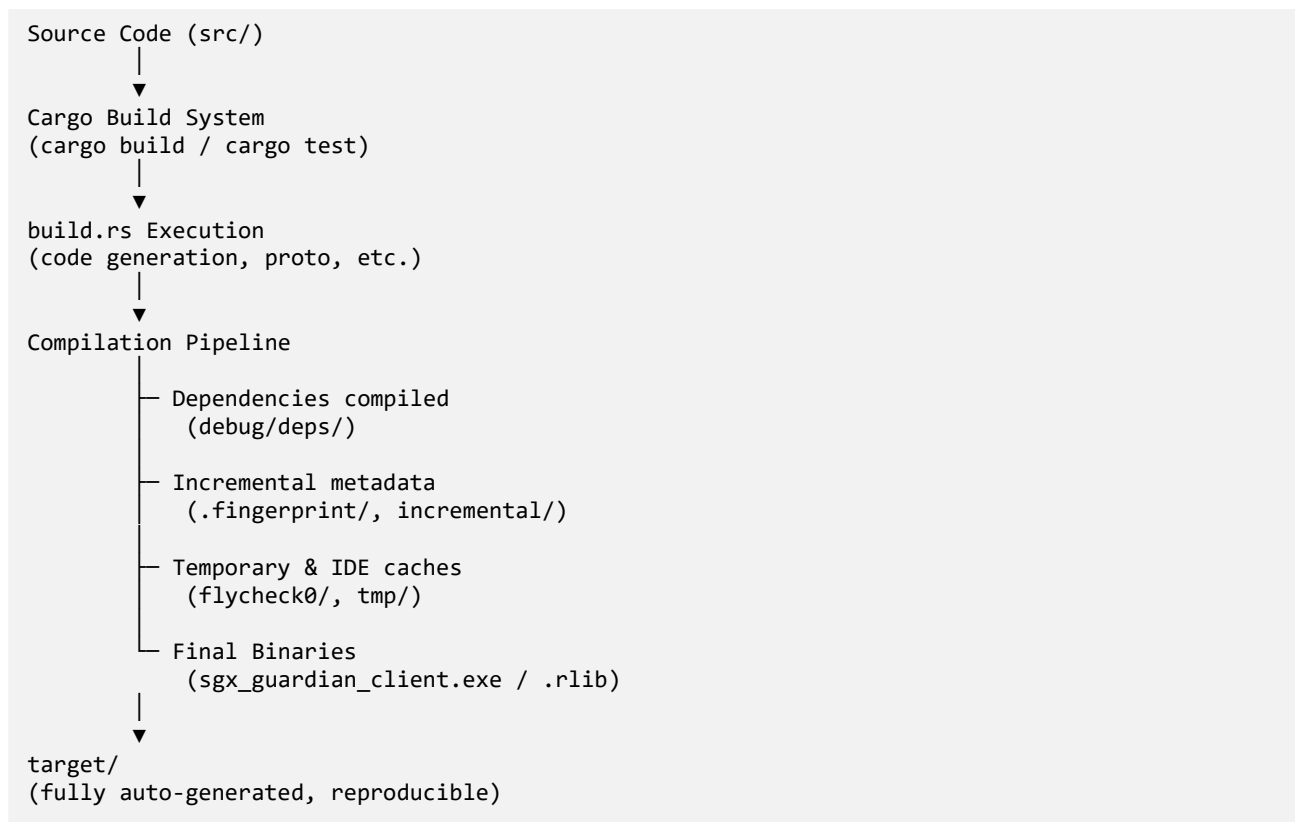
Concept-Clarifying Code Snippet

```
// main.rs (conceptual)
let km = KeyManager::load_or_generate(key_path)?;
let evidence =
  AttestationService::create_signed_evidence(&km,
  policy)?;
load_and_activate_policy(policy_sig)?;
start_server(addr, identity, ca_cert).await?;
enforcement::enforce_policy(active_policy)?;
```

2.12. target/ — Build Output & Compilation Workspace

The `src/` directory implements the entire trusted runtime of the SGX Guardian Client. It is responsible for identity, trust establishment, secure communication, policy enforcement, auditing, and observability.

Build & Artifact Flow (Diagram)



1. Purpose

The `target/` directory serves as Cargo's build workspace, storing all compiled binaries, intermediate artifacts, dependency outputs, and temporary compilation metadata generated during development and testing.

2. Responsibility

`target/` is responsible for:

- Holding debug build binaries
- Storing compiled dependency crates
- Preserving build script (`build.rs`) outputs
- Maintaining incremental compilation caches
- Supporting faster rebuilds and IDE tooling
- It has no runtime or architectural responsibility in production.

3. Inputs

Inputs that indirectly feed into `target/` include:

- Rust source code (`src/`)
- Dependency definitions (`Cargo.toml`, `Cargo.lock`)
- Build scripts (`build.rs`)
- Generated code (e.g., Protobuf output)
- Cargo commands:
 - `cargo clean`
 - `cargo build`
 - `cargo test`

4. Outputs

The directory produces temporary and reproducible outputs, such as:

- Debug binaries (sgx_guardian_client.exe, .rlib)
- Compiled dependency artifacts (debug/deps/)
- Incremental build metadata
- IDE helper artifacts (Flycheck / rust-analyzer)
- Debug symbols (.pdb on Windows)

These outputs are environment-specific and disposable.

5. Security / Constraints

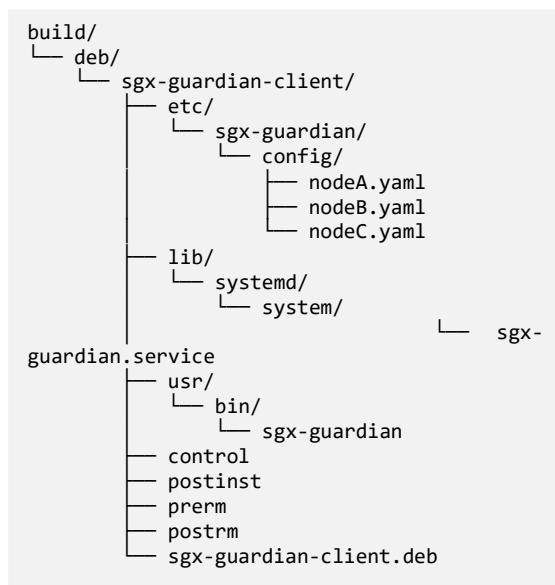
- target/ must never be committed to version control
- Debug artifacts may contain symbols and paths
- Contents are non-authoritative (rebuildable anytime)
- CI/CD pipelines generate separate, signed release artifacts
- Safe to delete without impacting source or configuration

2.13. build/ — Local Package Assembly Workspace

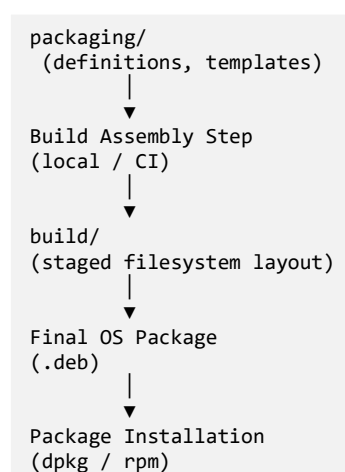
Important positioning:

- Build/ Is not part of runtime
- Build/ is part of packaging
- Build/ is just assembly output of local/CI package

Folder Structure (As Observed)



Build vs Packaging Flow (Diagram)



Key point:

build/ is an intermediate staging area, not a source of truth.

1. Purpose

The build/ directory acts as a temporary assembly workspace where packaging definitions are materialized into a filesystem layout before producing the final installable package.

2. Responsibility

build/ is responsible for:

- Holding staged filesystem structure (/etc, /usr/bin, /lib/systemd)
- Collecting binaries, configs, and service files
- Assembling control scripts alongside files
- Producing the final .deb or .rpm artifact
- It has no influence on runtime behavior.

3. Inputs

Inputs to the build/ directory include:

- Packaging definitions (packaging/)
- Release binaries (target/release/)
- systemd service files
- Configuration templates
- Policy files (if included)
- Build tooling (cargo-deb, rpmbuild, CI jobs)

4. Outputs

The build/ directory produces:

- Fully assembled OS package:
- sgx-guardian-client.deb
- Temporary staged directories:
- /etc/sgx-guardian/
- /usr/bin/
- /lib/systemd/system/

These outputs are ephemeral and reproducible.

5. Security / Constraints

- build/ contents are not trusted artifacts
- Safe to delete at any time
- No secrets should be stored here
- Runtime security does not depend on build directory
- Packaging remains valid even if build/ is corrupted

Concept-Clarifying Snippet

- packaging/ → defines what should exist
- build/ → shows how it will look when installed